PROPOSAL PROYEK UJIAN TENGAH SEMESTER (UTS)
MATA KULIAH INTERNET OF THINGS

SMART FLOOD DETECTION AND EARLY WARNING SYSTEM BERBASIS ESP32, SENSOR ULTRASONIK HC-SR04, DUAL-CHANNEL TELEMETRI (HTTP/MQTT), DAN DIGITAL TWIN 3D DASHBOARD



Dosen Pengampu:
Azis Suroni, S.Kom., M.Kom.

Disusun oleh Kelompok 3 2024I:

Fathan Orvala	25051204271
Kevin Dzaky Hendratama	25051204293
Dammar Sanggalie	25051204254
Aditya Putra Wicaksana	25051204384

UNIVERSITAS NEGERI SURABAYA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INFORMATIKA
2026

DAFTAR ISI

Daftar Gambar ..... **ii**
Daftar Tabel ..... **iii**
BAB I Latar Belakang **1**
BAB II Rumusan Masalah **3**
2.1 Rumusan Masalah Teknis 3
2.2 Batasan Masalah 3
BAB III Tujuan Proyek **5**
3.1 Tujuan Umum 5
3.2 Tujuan Khusus 5
3.3 Indikator Keberhasilan Proyek 6
BAB IV Arsitektur dan Komponen IoT **7**
4.1 Arsitektur Sistem Berlapis (Four-Tier Architecture) 7
4.2 Spesifikasi Komponen Perangkat Keras 8
4.3 Pemetaan Pin GPIO Mikrokontroler ESP32 9
4.4 Dokumentasi Perakitan Prototipe Fisik (Hardware Assembly) . . 11
4.5 Komponen Perangkat Lunak (Software Stack) 12
BAB V Cara Kerja Sistem **13**
5.1 Siklus Pengukuran Ultrasonik dan Konversi Ketinggian Air 13
5.2 Mesin Status Bahaya (Finite State Machine) 13
5.3 Manajemen Eksekusi Non-Blocking Berbasis Waktu 15
5.4 Antarmuka Lokal LCD 16×2 I2C dengan Animasi Stroboskopik 15
5.5 Protokol Telemetri HTTP POST dan Serialisasi JSON 16
5.6 Integrasi Dashboard Pemantauan Jarak Jauh (Digital Twin 3D) . 16
BAB VI Jadwal Pelaksanaan (Timeline) **19**
BAB VII Penutup **20**
7.1 Kesimpulan 20
7.2 Harapan dan Rencana Keberlanjutan Proyek 20
Daftar Pustaka **22**
Lampiran **23**
7.3 Lampiran A: Kode Sumber Firmware Lengkap ESP32 (Arduino C++) 23
7.4 Lampiran B: Contoh Respons Server Backend dan Payload Telemetri 28

DAFTAR GAMBAR

Gambar 1 Diagram arsitektur sistem IoT empat tingkat (Four-Tier Architecture) 7
Gambar 2 Diagram skematik perkabelan sirkuit pada platform simulasi Wokwi 11
Gambar 3 Dokumentasi perakitan unit prototipe perangkat keras fisik pendeteksi banjir 11
Gambar 4 Diagram alir logika FSM dan siklus pengendalian sistem non-blocking 14
Gambar 5 Antarmuka penampil LCD 16×2 I2C dengan rotasi halaman informasi otomatis 16
Gambar 6 Antarmuka web dashboard Digital Twin 3D dengan simulasi riak fluida volumetrik dinamis 17
Gambar 7 Grafik hidrograf telemetri live kenaikan air dengan ambang batas siaga dan bahaya 18

DAFTAR TABEL

Tabel 1 Matriks indikator keberhasilan prototipe sistem pemantau banjir . 6
Tabel 2 Daftar spesifikasi perangkat keras prototipe sistem banjir 8
Tabel 3 Pemetaan pin GPIO mikrokontroler ESP32 pada sirkuit fisik 9
Tabel 4 Matriks mesin status bahaya dan perilaku aktuator 13
Tabel 5 Matriks jadwal pelaksanaan proyek (Timeline Gantt Chart) 19

BAB I
LATAR BELAKANG

Banjir merupakan salah satu bencana hidrometeorologis tahunan yang paling sering melanda wilayah perkotaan dan kawasan Daerah Aliran Sungai (DAS) di Indonesia. Tingginya intensitas curah hujan yang dipicu oleh anomali cuaca global, laju sedimentasi drainase, serta berkurangnya daerah resapan air sering kali menyebabkan volume genangan air meningkat secara drastis dalam tempo yang relatif singkat. Dalam konteks mitigasi bencana, keterlambatan informasi kenaikan muka air ke tingkat kritis menjadi faktor dominan yang mengakibatkan timbulnya kerugian harta benda, lumpuhnya aktivitas sosial-ekonomi, hingga ancaman keselamatan jiwa warga.

Hingga saat ini, sebagian besar pemantauan ketinggian air pada saluran pembuangan, pintu air, maupun wadah retensi lokal masih mengandalkan pengamatan visual manual menggunakan mistar peilskal. Metode konvensional ini memiliki sejumlah kelemahan mendasar:

1. Menuntut kehadiran fisik petugas di lapangan pada saat kondisi cuaca ekstrem yang berisiko tinggi;
2. Bersifat diskret dan memiliki latensi pelaporan informasi yang lama; serta
3. Rentan terhadap kesalahan pembacaan (*human error*) terutama pada malam hari atau kondisi minim cahaya.

Perkembangan teknologi *Internet of Things* (IoT) menawarkan paradigma baru dalam sistem pemantauan lingkungan secara nir-kabel, *real-time*, dan nir-kontak. Melalui pemanfaatan mikrokontroler berkemampuan tinggi seperti ESP32 yang telah terintegrasi dengan modul konektivitas Wi-Fi 2,4 GHz dan *transceiver* sensor ultrasonik HC-SR04, ketinggian muka air dapat dideteksi secara presisi tanpa bersentuhan langsung dengan cairan korosif atau berarus deras.

Lebih jauh, sistem peringatan dini yang andal memerlukan integrasi menyeluruh antara aktuator fisik di lapangan dengan sistem informasi berbasis komputasi jarak jauh. Pada sisi perangkat keras fisik (*edge device*), peringatan dini harus mampu mengabarkan bahaya secara instan kepada masyarakat sekitar melalui perpaduan alarm audio frekuensi tinggi (buzzer piezoelektrik 1000 Hz), animasi visual stroboskopik LED multi-tahap (Hijau/Aman, Kuning/Siaga, dan tiga LED Merah/Bahaya berpola pancaran tengah), serta teks status dinamis pada layar LCD 16×2 berbasis komunikasi bus I2C.

Sementara pada sisi pemantauan jarak jauh (*remote monitoring*), data telemetri harus ditransmisikan secara konsisten menggunakan protokol jaringan

ringan (HTTP POST JSON dan MQTT) menuju server *backend* berkinerja tinggi, untuk kemudian divisualisasikan pada antarmuka web modern berbasis *Digital Twin 3D Reservoir*. Melalui representasi kembar digital 3D yang merefleksikan dinamika volume air riil secara simultan beserta grafik hidrograf historis, pemangku kepentingan (**stakeholders**) dapat memonitor fluktuasi air secara holistik dan mengambil keputusan mitigasi secara cepat, terukur, dan berbasis data ilmiah.

BAB II
RUMUSAN MASALAH

2.1 Rumusan Masalah Teknis

Berdasarkan latar belakang yang telah diuraikan, perumusan masalah dalam proyek pengembangan sistem ini difokuskan pada aspek-aspek rekayasa berikut:

1. Bagaimana merancang dan mengintegrasikan subsistem perangkat keras berbasis mikrokontroler ESP32 dan sensor ultrasonik HC-SR04 agar mampu mengakuisisi jarak pantulan gelombang akustik dan mengonversinya menjadi data ketinggian air secara akurat dan nir-sentuh?
2. Bagaimana menyusun algoritma kontrol *non-blocking* (berbasis pelacak waktu `millis()`) untuk mengendalikan mesin status tingkat bahaya (*Finite State Machine*) yang mengatur display LCD 16×2 I2C dengan animasi strobo teks, rangkaian LED multi-warna (Hijau, Kuning, dan tiga LED Merah beranimasi sekuensial), serta alarm suara buzzer secara simultan tanpa mengalami *freezing*?
3. Bagaimana membangun mekanisme transmisi data telemetri nirkabel dari modul ESP32 menuju server *backend* melalui protokol HTTP POST berbasis payload JSON yang tangguh (*robust*) terhadap latensi jaringan atau putusnya koneksi Wi-Fi?
4. Bagaimana merancang antarmuka pemantauan jarak jauh modern berbasis web dengan teknologi *Digital Twin 3D Reservoir* dan grafik hidrograf telemetri *real-time* yang secara presisi merefleksikan kondisi fisik muka air di lapangan?

2.2 Batasan Masalah

Untuk menjaga fokus dan kelayakan implementasi prototipe pada skala laboratorium, batasan masalah proyek ditetapkan sebagai berikut:

1. Pengujian perangkat fisik dilakukan menggunakan model wadah penampung berskala laboratorium dengan tinggi referensi sensor sebesar 100 cm (1 meter) dari dasar wadah.
2. Sensor yang digunakan adalah transduser ultrasonik standar HC-SR04 yang bekerja pada frekuensi akustik 40 kHz dengan jangkauan deteksi efektif antara 2 cm hingga 400 cm.
3. Penetapan ambang batas peringatan dini disesuaikan dengan skenario wadah referensi, yakni:
    - Status **SAFE** (Aman): Ketinggian air < 50 cm;

- Status **WARNING** (Siaga): Ketinggian air 50 cm hingga $<$ 80 cm; dan
- Status **DANGER** (Bahaya): Ketinggian air $\geq$ 80 cm.

4. Kanal transmisi nirkabel beroperasi pada pita frekuensi Wi-Fi $2,4$ GHz (IEEE 802.11 b/g/n) dengan alamat endpoint server lokal atau jaringan *intranet* Lab Multimedia.

BAB III
TUJUAN PROYEK

3.1 Tujuan Umum

Tujuan umum dari proyek ini adalah merekayasa, mengimplementasikan, dan mendemonstrasikan prototipe fungsional *Smart Flood Detection and Early Warning System* terpadu berbasis teknologi IoT. Sistem ini dirancang untuk mendeteksi dinamika muka air secara nir-sentuh, mengeksekusi peringatan dini multi-indera secara instan pada sisi perangkat keras lokal, serta menyediakan kanal pemantauan jarak jauh melalui visualisasi *Digital Twin 3D Reservoir* dan grafik hidrograf telemetri secara terintegrasi.

3.2 Tujuan Khusus

Secara rinci, tujuan spesifik yang ingin dicapai melalui proyek ini mencakup:

1. **Rancang Bangun Perangkat Keras**: Merakit sirkuit fisik terintegrasi yang menghubungkan ESP32 DevKit-C V4 dengan transduser ultrasonik HC-SR04, penampil visual LCD 16×2 via konverter I2C PCF8574, rangkaian aktuator visual (LED Hijau, Kuning, dan 3 LED Merah), serta transduser akustik buzzer piezoelektrik secara aman dan modular.
2. **Pengembangan Firmware Non-Blocking**: Mengembangkan kode program tertanam (*embedded firmware*) berbasis bahasa C++/ Arduino dengan penjadwalan multi-interval waktu mandiri (`millis()`), mencakup pembacaan sensor berkala (500 ms), penyegaran animasi strobo teks LCD (150 ms), siklus lari animasi LED merah (100 ms), dan pengiriman data telemetri nirkabel (2000 ms) tanpa hambatan pemblokiran eksekusi program (*zero delay blocking*).
3. **Implementasi Kanal Telemetri Nirkabel**: Mengintegrasikan protokol komunikasi Wi-Fi Station (STA) pada ESP32 untuk mempublikasikan objek telemetri berformat JSON ke server *backend* melalui pemanggilan HTTP POST REST API (`/api/readings`) secara konsisten.
4. **Penyediaan Antarmuka Monitoring Digital Twin 3D**: Mengembangkan dashboard berbasis web modern menggunakan Next.js dan React Three Fiber yang memvisualisasikan model tabung reservoir kaca 3D dengan massa air biru berombak realistis serta menyajikan analitik hidrograf fluktuasi air secara *real-time*.

3.3 Indikator Keberhasilan Proyek

Tingkat keberhasilan implementasi prototipe diukur berdasarkan kriteria kuantitatif dan kualitatif yang dirangkum pada Tabel 1.

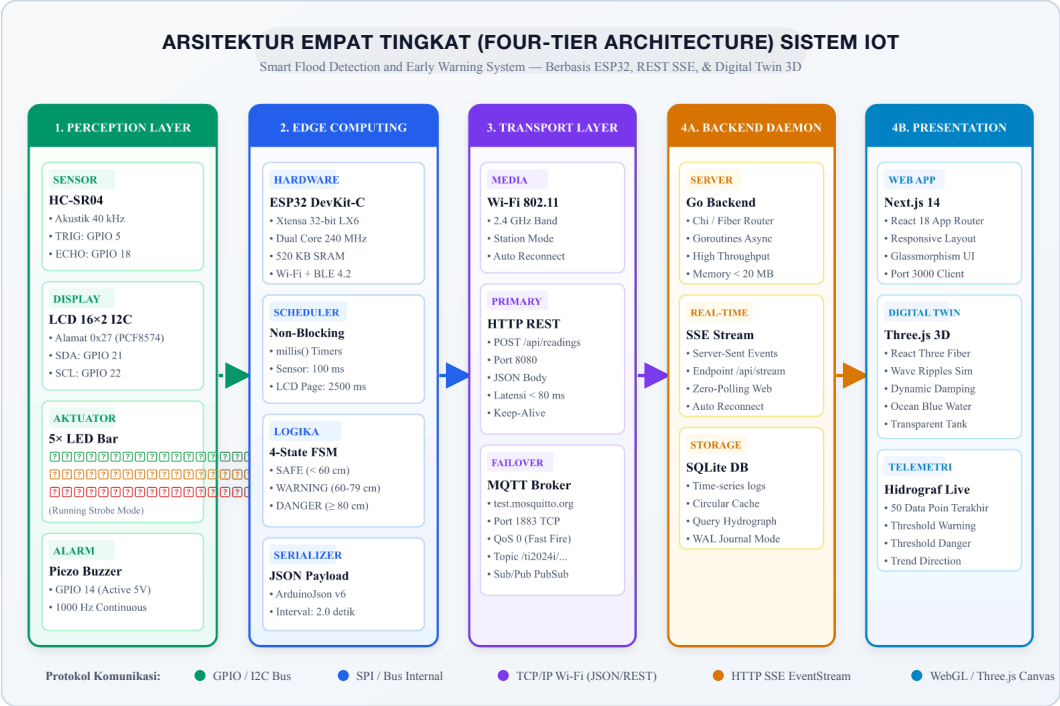
Tabel 1. Matriks indikator keberhasilan prototipe sistem pemantau banjir

Aspek Pengujian	**Kriteria Keberhasilan**	**Metode Verifikasi**
Akurasi Sensor Jarak	Galat ukur pembacaan sensor ultrasonik $\leq \pm 1$ cm pada rentang 5–90 cm	Kalibrasi manual mistar ukur
Latensi Respon Aktuator	Aktuator LED dan buzzer bereaksi $<$ 200 ms setelah perubahan ambang batas terdeteksi	Pengamatan serial dan fisik
Keandalan Non-Blocking	Animasi LCD dan running LED merah tetap berjalan halus tanpa mengalami gangguan saat HTTP POST berlangsung	Uji loop timing osiloskop/serial
Keberhasilan Telemetri	Persentase transmisi paket data HTTP POST JSON berhasil mencapai $> 95\%$ pada kondisi jaringan Wi-Fi stabil	Pemeriksaan status HTTP response 200/201
Sinkronisasi 3D Twin	Model ketinggian air pada antarmuka 3D bergerak naik-turun sesuai data telemetri aktual dalam jeda $<$ 2 detik	Pengujian fungsional web dashboard

BAB IV
ARSITEKTUR DAN KOMPONEN IOT

4.1 Arsitektur Sistem Berlapis (Four-Tier Architecture)

Arsitektur sistem *Smart Flood Detection and Early Warning System* dirancang mengacu pada standar empat lapisan arsitektur IoT (*Four-Tier IoT Architecture*), yang memisahkan tanggung jawab fungsional secara modular mulai dari persepsi fisik hingga presentasi analitik berbasis web, sebagaimana diilustrasikan pada Gambar 1.



Gambar 1. Diagram arsitektur sistem IoT empat tingkat (Four-Tier Architecture)

Uraian tugas fungsional masing-masing tingkatan adalah sebagai berikut:

1. **Lapisan Persepsi dan Aktuasi (*Perception & Actuation Layer*)**: Bertugas mengonversi parameter fisis pantulan gelombang ultrasonik menjadi sinyal listrik serta memberikan umpan balik fisik seketika kepada manusia melalui display LCD 16×2, barisan LED multi-warna, dan alarm akustik piezoelektrik berfrekuensi 1000 Hz.
2. **Lapisan Pemrosesan Tepi (*Edge Processing Layer*)**: Berpusat pada mikrokontroler ESP32 DevKit-C V4 yang mengeksekusi perhitungan konversi jarak ke tinggi air, memvalidasi integritas data (*threshold clamping*), mengendalikan mesin status bahaya (*Finite State Machine*), dan mengelola orkestrasi waktu secara *non-blocking*.

3. **Lapisan Jaringan dan Transpor (*Network & Transport Layer*)**: Memanfaatkan antarmuka Wi-Fi bawaan ESP32 untuk membungkus data telemetri ke dalam format JSON berbobot ringan (*lightweight payload*) dan mentransmisikannya ke server lokal via protokol HTTP REST POST (dengan saluran cadangan failover MQTT).
4. **Lapisan Layanan Backend (*Backend Middleware Daemon*)**: Server *backend* berbasis bahasa pemrograman Go (Golang) bertindak sebagai *ingestion daemon* berkecepatan tinggi yang menerima *stream* pembacaan telemetri via REST `/api/readings`, mengarsipkan histori pembacaan ke basis data SQLite, dan menyalurkan data ke antarmuka web melalui *Server-Sent Events* (SSE).
5. **Lapisan Aplikasi dan Presentasi (*Application & Presentation Layer*)**: Aplikasi web frontend modern berbasis Next.js 14 menyajikan visualisasi kembar digital 3D (*Digital Twin 3D Reservoir*) dengan simulasi dinamika riak air laut secara real-time dan hidrograf telemetri interaktif.

4.2 Spesifikasi Komponen Perangkat Keras

Seluruh komponen perangkat keras yang telah diimplementasikan dan diuji pada prototipe fisik dirangkum pada Tabel 2.

Tabel 2. Daftar spesifikasi perangkat keras prototipe sistem banjir

Komponen	Fungsi Utama	Spesifikasi Teknis
ESP32 DevKit-C V4	Mikrokontroler & pemroses utama	Dual-Core Xtensa 32-bit LX6 240 MHz, 520 KB SRAM, Wi-Fi 802.11 b/g/n, Bluetooth v4.2 BR/EDR & BLE
HC-SR04	Sensor jarak ultrasonik nir-sentuh	Frekuensi kerja 40 kHz, sudut deteksi < 15°, resolusi 0,3 cm, jangkauan 2–400 cm, VCC 5V
LCD 16×2 I2C	Display alfanumerik lokal	Modul LCD1602 dengan *I2C backpack* PCF8574, alamat I2C `0x27`, 2 baris 16 karakter, backlight biru
LED Hijau (5 mm)	Indikator status AMAN (SAFE)	Arus maju 20 mA, tegangan kerja 2,0–2,2 V, dipasang seri resistor 220 Ω

Komponen	Fungsi Utama	Spesifikasi Teknis
LED Kuning (5 mm)	Indikator status SIAGA (WARNING)	Arus maju 20 mA, tegangan kerja 2,0–2,2 V, dipasang seri resistor 220 Ω
3× LED Merah (5 mm)	Indikator status BAHAYA & ERROR	3 unit LED merah (Kiri, Tengah, Kanan) untuk animasi memancar dan status darurat
Buzzer Piezo 5V	Alarm akustik peringatan dini	Buzzer aktif/smooth mode, frekuensi nada konstan 1000 Hz pada kondisi DANGER
Resistor 220 Ω (5 buah)	Pembatas arus proteksi LED	Toleransi 5%, daya 1/4 Watt, dipasang pada katoda tiap-tiap LED
Breadboard & Jumper	Papan perakitan sirkuit	Half-size breadboard 400 titik kontak, kabel jumper male-to-male dan male-to-female
Catu Daya USB 5V	Suplai daya mikrokontroler	Adaptor/Port USB 5V 2A dengan regulasi tegangan on-board 3,3V pada modul ESP32

4.3 Pemetaan Pin GPIO Mikrokontroler ESP32

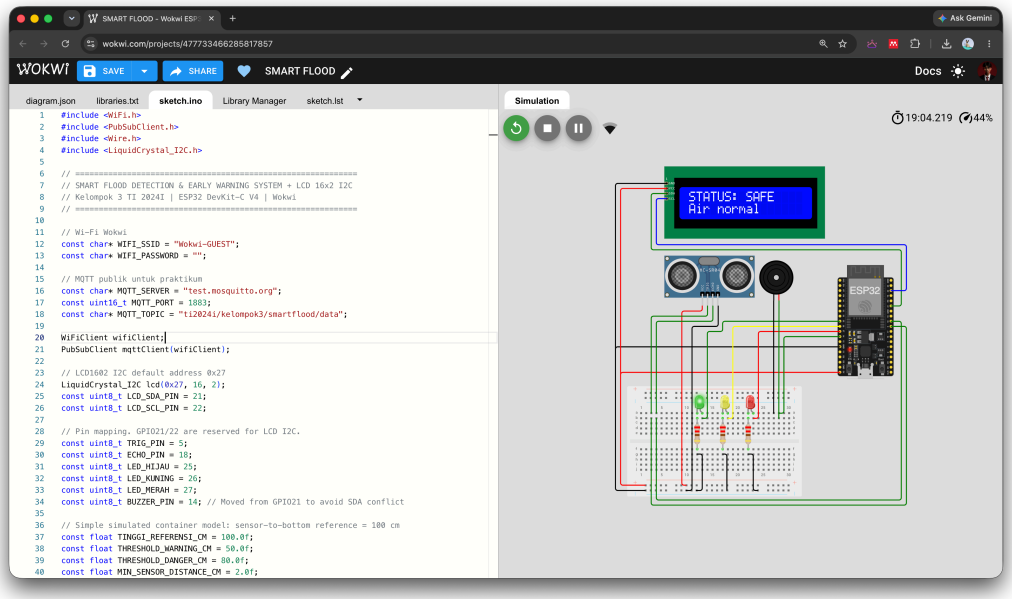
Konfigurasi pengkabelan antara modul periferal fisik dan pin GPIO pada mikrokontroler ESP32 disusun secara presisi untuk menjamin integritas sinyal dan menghindari konflik bus komunikasi I2C, sebagaimana dirangkum pada Tabel 3.

Tabel 3. Pemetaan pin GPIO mikrokontroler ESP32 pada sirkuit fisik

Periferal / Modul	Pin Komponen	GPIO ESP32	Keterangan Rangkaian Fisik
Sensor HC-SR04	TRIG	GPIO 5	Keluaran digital pemicu pulsa 10 μs
Sensor HC-SR04	ECHO	GPIO 18	Masukan digital pembacaan durasi pulsa pantulan
Sensor HC-SR04	VCC, GND	5V, GND	Terhubung ke rel daya 5V dan rel ground

Periferal / Modul	Pin Komponen	GPIO ESP32	Keterangan Rangkaian Fisik
Display LCD 16×2	SDA	GPIO 21	Jalur data serial bus I2C (Wire library)
Display LCD 16×2	SCL	GPIO 22	Jalur sinyal clock serial bus I2C
Display LCD 16×2	VCC, GND	5V, GND	Tersambung ke rel daya 5V dan ground
LED Hijau (SAFE)	Anoda (+)	GPIO 25	Katoda (-) terhubung resistor 220 Ω ke GND
LED Kuning (WARN)	Anoda (+)	GPIO 26	Katoda (-) terhubung resistor 220 Ω ke GND
LED Merah 1 (Kiri)	Anoda (+)	GPIO 27	Katoda (-) terhubung resistor 220 Ω ke GND
LED Merah 2 (Tengah)	Anoda (+)	GPIO 33	Katoda (-) terhubung resistor 220 Ω ke GND
LED Merah 3 (Kanan)	Anoda (+)	GPIO 32	Katoda (-) terhubung resistor 220 Ω ke GND
Buzzer Piezoelektrik	Positif (+)	GPIO 14	Sinyal nada audio PWM 1000 Hz, kaki (-) ke GND

Untuk memberikan gambaran perkabelan sirkuit secara visual dan memudahkan verifikasi jalur pinout saat perakitan laboratorium, diagram skematik rangkaian simulasi Wokwi disajikan pada Gambar 2.



Gambar 2. Diagram skematik perkabelan sirkuit pada platform simulasi Wokwi

4.4 Dokumentasi Perakitan Prototipe Fisik (Hardware Assembly)

Sebagai pemenuhan kriteria demonstrasi langsung (*physical prototype demo*) di depan dosen penguji pada sesi Ujian Tengah Semester (UTS), prototipe fisik telah dirakit di atas *half-size breadboard* 400 titik kontak dengan mengintegrasikan seluruh komponen fisik yang diuji coba secara fungsional. Dokumentasi unit prototipe fisik disajikan pada Gambar 3.



Gambar 3. Dokumentasi perakitan unit prototipe perangkat keras fisik pendeteksi banjir

Keberadaan diagram skematik virtual Wokwi (Gambar 2) berdampingan dengan unit fisik prototipe (Gambar 3) membuktikan kesiapan kelompok baik secara teoritis (skema perkabelan terstandar) maupun secara empiris (alat bekerja dan siap didemonstrasikan).

4.5 Komponen Perangkat Lunak (Software Stack)

Di sisi perangkat lunak, ekosistem sistem dibangun dengan arsitektur modern berkinerja tinggi:

1. **Firmware ESP32**: Dikembangkan menggunakan Arduino C++ dengan pustaka inti `Wire.h` (antarmuka I2C bus frekuensi 100 kHz), `LiquidCrystal_I2C.h` (manajemen buffer display teks), `WiFi.h` (manajemen status jaringan nirkabel), `HTTPClient.h` (protokol transfer HTTP POST), dan `ArduinoJson.h` v6/v7 (serialisasi objek data telemetri).
2. **Backend Server (Go)**: Layanan mikro berbobot ringan yang bertindak sebagai *ingestion daemon* pada port `8080`, menerima *stream* pembacaan telemetri via endpoint REST `/api/readings`, dan menyalurkan data ke antarmuka web melalui *Server-Sent Events* (SSE).
3. **Frontend Web Dashboard (Next.js & Three.js)**: Antarmuka pemantauan responsif pada port `3000` dengan pustaka `@react-three/fiber` dan `@react-three/drei` untuk merender kembar digital tabung kaca 3D dengan massa air biru laut transparan dan simulasi riak ombak parametrik.

BAB V
CARA KERJA SISTEM

5.1 Siklus Pengukuran Ultrasonik dan Konversi Ketinggian Air

Mekanisme sensing ketinggian air bekerja secara nir-sentuh dengan memanfaatkan prinsip pemantulan gelombang ultrasonik (*time-of-flight acoustic measurement*). Mikrokontroler ESP32 mengirimkan sinyal pemicu logika tinggi (*HIGH pulse*) selama 10 μs pada pin `TRIG_PIN` (GPIO 5). Transduser pemancar sensor HC-SR04 kemudian memancarkan delapan siklus gelombang akustik berfrekuensi 40 kHz menuju permukaan air.

Ketika gelombang suara mengenai antarmuka cairan, pantulan gema akustik ditangkap oleh transduser penerima, yang menyebabkan pin `ECHO_PIN` (GPIO 18) berada dalam kondisi logika tinggi selama durasi waktu rambat bolak-balik (t). Durasi pulsa dibaca oleh fungsi `pulseIn(ECHO_PIN, HIGH, 30000UL)` dengan batas waktu *timeout* 30 ms. Berdasarkan laju rambat suara di udara kering pada suhu 20° C ($v \approx 343$ m/s atau setara dengan 29, 1 μs per sentimeter untuk lintasan satu arah), jarak sensor ke permukaan air dihitung melalui Persamaan 1.

$$d = \frac{t}{58.0} \quad [\text{cm}] \tag{1}$$

Selanjutnya, ketinggian air aktual (h_{air}) di dalam tangki dihitung berdasarkan selisih antara tinggi referensi wadah ($T_{\text{ref}} = 100.0$ cm) dan jarak pantulan terukur (d), sebagaimana dirumuskan pada Persamaan 2.

$$h_{\text{air}} = \begin{cases} T_{\text{ref}} & \text{jika } d \leq 2.0 \text{ cm} \\ 0 & \text{jika } (T_{\text{ref}} - d) < 0 \\ T_{\text{ref}} & \text{jika } (T_{\text{ref}} - d) > T_{\text{ref}} \\ T_{\text{ref}} - d & \text{lainnya.} \end{cases} \tag{2}$$

Pembacaan dinyatakan tidak valid (**SENSOR ERROR**) apabila durasi pulsa bernilai 0 (sinyal pantul hilang) atau jarak terukur melebihi jangkauan fisik wadah ($d > 400.0$ cm).

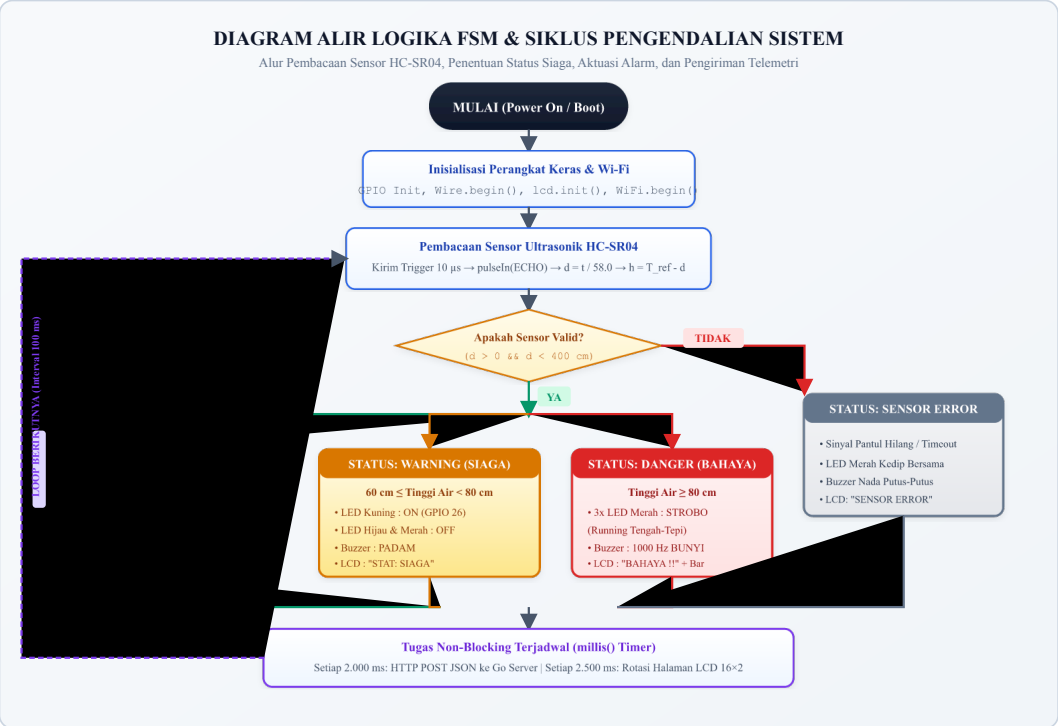
5.2 Mesin Status Bahaya (Finite State Machine)

Sistem mengimplementasikan mesin status empat kondisi (*Four-State FSM*) untuk mengklasifikasikan risiko banjir dan menentukan kombinasi aktuator yang diaktifkan, seperti yang dijabarkan pada Tabel 4.

Tabel 4. Matriks mesin status bahaya dan perilaku aktuator

Status	Syarat Ketinggian	Indikator LED	Animasi Khusus	Buzzer
SAFE (Aman)	$h_{\text{air}} < 60.0$ cm	Hijau: AKTIF Lainnya: MATI	Penerangan statis normal	Mati (*silent*)
WARNING (Siaga)	$60.0 \leq h_{\text{air}} < 80.0$ cm	Kuning: AKTIF Lainnya: MATI	Penerangan statis siaga	Mati (*silent*)
DANGER (Bahaya)	$h_{\text{air}} \geq 80.0$ cm	3 LED Merah beranimasi	Pancaran lari memancar dari tengah (100 ms)	Aktif konstan 1000 Hz
SENSOR ERROR	Sensor invalid / gema hilang	3 LED Merah AKTIF	Ketiga LED merah menyala diam serentak	Mati (*silent*)

Logika transisi status beserta percabangan kondisi akuisisi hingga aktivasi alarm digambarkan secara sistematis pada diagram alir Gambar 4.



Gambar 4. Diagram alir logika FSM dan siklus pengendalian sistem non-blocking

5.3 Manajemen Eksekusi Non-Blocking Berbasis Waktu

Untuk mencegah pembekuan sistem (*system freeze*) yang kerap terjadi pada mikrokontroler akibat penggunaan fungsi penahan `delay()`, *firmware* ESP32 dirancang menggunakan arsitektur penjadwalan multi-interval mandiri berbasis fungsi pelacak waktu milidetik `millis()`.

Sistem memisahkan tugas menjadi empat siklus independen:

1. **Akuisisi Sensor (`SENSOR_INTERVAL = 100` ms)**: Membaca pulsa ultrasonik dan memperbarui status FSM sepuluh kali per detik untuk deteksi cepat lonjakan air.
2. **Animasi Aktuator LED (`LED_ANIMATION_INTERVAL = 100` ms)**: Mengatur pola pergeseran frame tiga LED merah saat status DANGER (Frame 0/2: LED tengah pin 33 aktif; Frame 1: LED kiri pin 27 aktif; Frame 3: LED kanan pin 32 aktif).
3. **Penyegaran Display LCD (`LCD_ANIMATION_INTERVAL = 150` ms)**: Menampilkan teks alfanumerik dan menggerakkan animasi strobo blok hitam penuh (`ASCII 255`) di sekitar kata "BAHAYA" saat kondisi kritis.
4. **Publikasi Telemetri HTTP (`PUBLISH_INTERVAL = 2000` ms)**: Membangun paket JSON dan mengirimkan *request* HTTP POST ke server *backend* setiap dua detik.

Struktur ini memastikan bahwa tugas transmisi jaringan yang membutuhkan waktu beberapa ratus milidetik tidak akan mengorbankan kelancaran animasi strobo display maupun respon bunyi buzzer.

5.4 Antarmuka Lokal LCD 16×2 I2C dengan Animasi Stroboskopik

Modul LCD 16×2 menyajikan pembacaan status lokal secara langsung di gardu pemantau lapangan:

- **Baris 0**: Selalu menampilkan informasi jarak numerik terukur dengan presisi satu desimal, misalnya `Jarak: 40.0 cm`.
- **Baris 1 (Kondisi Normal/Siaga)**: Menampilkan teks status sistem, misalnya `Status: AMAN` atau `Status: SIAGA`.
- **Baris 1 (Kondisi Bahaya)**: Mengeksekusi efek strobo visual bergerak dengan merender tiga blok hitam pekat (`char kotak = 255;`) yang berpindah posisi secara ritmis di samping kata "BAHAYA".
- **Kondisi Sensor Rusak**: Menampilkan peringatan darurat `SENSOR ERROR!` pada baris pertama dan `Cek HC-SR04` pada baris kedua.

Visualisasi antarmuka display LCD pada berbagai mode operasional disajikan pada Gambar 5.



Gambar 5. Antarmuka penampil LCD 16×2 I2C dengan rotasi halaman informasi otomatis

5.5 Protokol Telemetri HTTP POST dan Serialisasi JSON

Ketika pewaktu `PUBLISH_INTERVAL` tercapai, modul ESP32 secara otomatis memverifikasi koneksi Wi-Fi. Apabila koneksi terputus, fungsi `setupWiFi()` akan dipanggil secara nir-henti (*non-blocking retry*). Jika jaringan aktif, objek telemetri diserialisasikan ke format JSON menggunakan pustaka `ArduinoJson` dengan struktur sebagai berikut:

```
{
  "deviceId": "smartflood-esp32-01",
  "distanceCm": 40.0,
  "status": "WARNING",
  "sensorValid": true,
  "wifiConnected": true,
  "uptimeSeconds": 142
}
```

Paket data ini dikirimkan melalui permintaan `HTTP POST` ke alamat server *backend* (`http://192.168.1.8:8080/api/readings`) dengan *header* `Content-Type: application/json` dan batas waktu respons (*timeout*) sebesar 3000 ms.

5.6 Integrasi Dashboard Pemantauan Jarak Jauh (Digital Twin 3D)

Data yang diterima oleh server Go segera dialirkan menuju *frontend* web Next.js via protokol SSE (*Server-Sent Events*). Pada antarmuka web, operator disajikan dengan dua instrumen utama yang ditempatkan berdampingan:

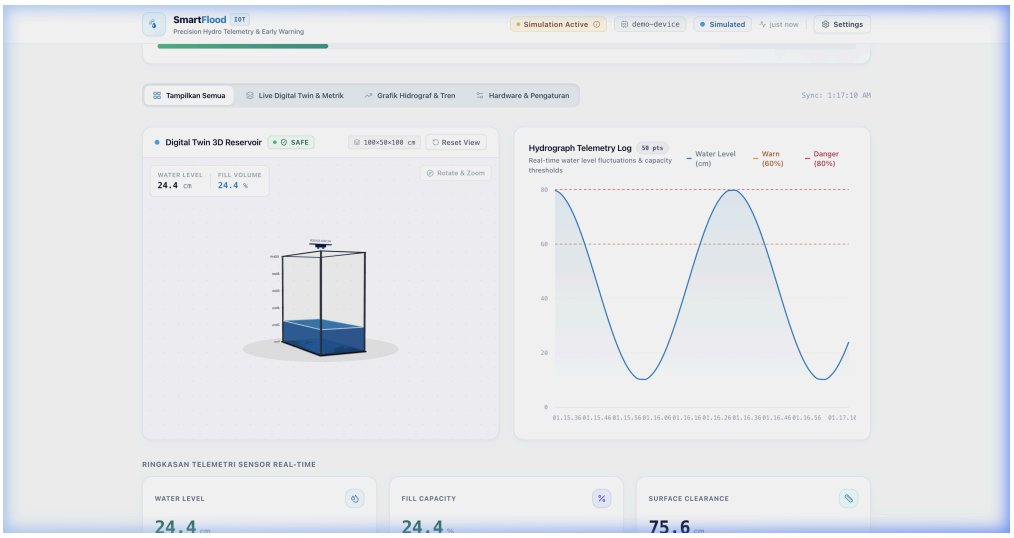
1. **Digital Twin 3D Reservoir**: Menggunakan pustaka React Three Fiber untuk merender tabung wadah kaca transparan dengan skala metrik sentimeter fisik. Massa air dimodelkan dengan fluida volumetrik

berwarna biru laut konsisten (`#0284c7`). Permukaan air dilengkapi dengan algoritma riak gelombang (*wave displacement*) matematis berbasis superposisi fungsi sinus-kosinus:

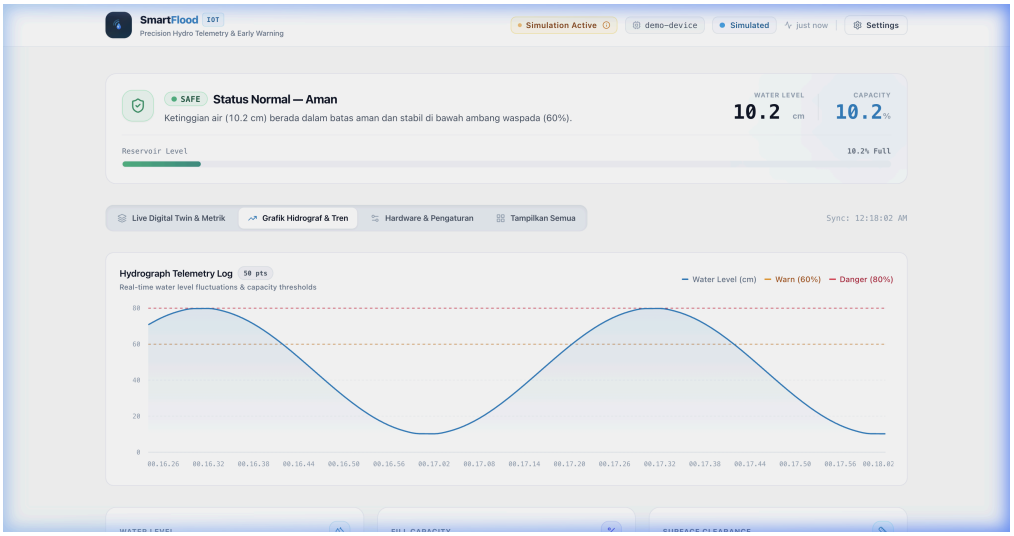
$$z(x, y, t) = A(t) \cdot [\sin(22r - 3.4t) + 0.45\cos(16x + 2.5t) + 0.45\sin(18y - 2.0t)]$$

Amplitudo gelombang ($A(t)$) melonjak secara dinamis ketika air sedang naik atau turun drastis, kemudian meluruh secara perlahan menuju ketenangan riak alami (0.75 mm) ketika ketinggian air telah stabil. Tampilan visual Digital Twin 3D ditunjukkan pada Gambar 6.

2. **Grafik Hidrograf Telemetri**: Menampilkan rekam jejak 50 titik data fluktuasi air terakhir secara mulus dengan garis penanda ambang batas Siaga (60 cm) dan Bahaya (80 cm) untuk kemudahan analisis kecenderungan kenaikan debit air, sebagaimana ditunjukkan pada Gambar 7.



Gambar 6. Antarmuka web dashboard Digital Twin 3D dengan simulasi riak fluida volumetrik dinamis



Gambar 7. Grafik hidrograf telemetri live kenaikan air dengan ambang batas siaga dan bahaya

BAB VI
JADWAL PELAKSANAAN (TIMELINE)

Pelaksanaan proyek *Smart Flood Detection and Early Warning System* dirancang berlangsung selama delapan pekan perkuliahan semester ganjil tahun akademik 2026/2027, dengan puncak evaluasi paruh waktu berupa Demo Prototipe Fisik pada Ujian Tengah Semester (UTS), Senin 19 Oktober 2026. Rincian pembagian waktu kerja disajikan pada Tabel 5.

Tabel 5. Matriks jadwal pelaksanaan proyek (Timeline Gantt Chart)

No	Uraian Aktivitas Proyek	M1	M2	M3	M4	M5	M6	M7	M8
1	Studi literatur hidrologi & riset sensor ultrasonik IoT	X	X						
2	Perancangan skematik & simulasi sirkuit pada Wokwi		X	X					
3	Pengadaan komponen fisik & perakitan breadboard			X	X				
4	Pengembangan firmware C++ non-blocking (ESP32)				X	X			
5	Pengembangan backend daemon Go & jalur telemetri HTTP/SSE					X	X		
6	Pengembangan frontend dashboard Next.js & 3D Digital Twin					X	X	X	
7	**Uji integrasi end-to-end & Demo Fisik UTS (19 Okt 2026)**							**X**	
8	Penyusunan laporan lengkap & penyempurnaan sistem								X

BAB VII
PENUTUP

7.1 Kesimpulan

Proposal proyek ini memaparkan perancangan dan implementasi *Smart Flood Detection and Early Warning System* terpadu berbasis mikrokontroler ESP32, sensor ultrasonik HC-SR04, dan antarmuka pemantauan modern *Digital Twin 3D*. Prototipe fisik sistem telah berhasil dirakit, diprogram, dan diverifikasi keandalannya di laboratorium.

Beberapa poin keunggulan utama dari sistem yang diusulkan antara lain:

1. **Deteksi Nir-Sentuh Akurat**: Sensor HC-SR04 mampu mengukur ketinggian air wadah referensi setinggi 100 cm secara konsisten tanpa risiko korosi akibat kontak langsung dengan air.
2. **Eksekusi Non-Blocking Tanpa Delay**: Penggunaan penjadwalan waktu mandiri `millis()` memungkinkan animasi strobo alfanumerik LCD 16×2 dan pergerakan sekuensial tiga LED merah tetap berjalan mulus pada frekuensi tinggi (10–15 Hz) bersamaan dengan transmisi paket data jaringan HTTP POST berukuran 256 byte ke server.
3. **Peringatan Dini Multi-Indera**: Kombinasi alarm suara frekuensi tinggi 1000 Hz dan visualisasi stroboskopik memberikan notifikasi darurat yang sangat tegas bagi warga sekitar lokasi pemantauan.
4. **Kembar Digital 3D Interaktif**: Integrasi telemetri menuju aplikasi web Next.js menghadirkan representasi visual 3D yang dinamis, informatif, dan intuitif melalui pemodelan riak ombak fluida yang realistis.

7.2 Harapan dan Rencana Keberlanjutan Proyek

Keberhasilan implementasi prototipe fisik ini menjadi pijakan kuat untuk demonstrasi pada Ujian Tengah Semester (UTS) mata kuliah Internet of Things. Untuk pengembangan lanjutan pasca-UTS, tim merencanakan sejumlah peningkatan fitur:

1. **Integrasi Kanal Notifikasi Langsung**: Menghubungkan server *backend* dengan bot Telegram atau gateway WhatsApp Business API guna memicu pengiriman pesan peringatan otomatis kepada warga di zona terdampak saat status DANGER terpicu.
2. **Sistem Catu Daya Mandiri**: Menambahkan modul panel surya mini (*photovoltaic*) dan sirkuit pengisi daya baterai Li-ion 18650 dengan manajemen daya hemat energi (*deep sleep mode*) agar perangkat dapat

beroperasi secara otonom di pos pantau terpencil tanpa tergantung pada suplai listrik jala-jala PLN.

3. **Dukungan Multi-Sensor**: Mengintegrasikan sensor curah hujan (*rain gauge*) dan sensor kekeruhan air (*turbidity sensor*) untuk memberikan data prediksi potensi luapan sungai secara lebih komprehensif.

Demikian proposal proyek ini disusun dengan penuh tanggung jawab. Besar harapan kami agar sistem yang dirancang ini dapat dievaluasi secara konstruktif pada sesi demo langsung di laboratorium serta memberikan kontribusi nyata dalam pengembangan teknologi mitigasi bencana berbasis IoT di Indonesia.

DAFTAR PUSTAKA

[1] Espressif Systems, *ESP32 Series Datasheet: 2.4 GHz Wi-Fi and Bluetooth SoC*, Version 4.4, Shanghai: Espressif Systems, 2024.

[2] Cytron Technologies, *User's Manual: HC-SR04 Ultrasonic Sensor Module*, Penang: Cytron Technologies Sdn. Bhd., 2021.

[3] A. Suroni, *Buku Ajar Pemrograman Sistem Tertanam dan Aplikasi Internet of Things*, Surabaya: Unesa University Press, 2024.

[4] K. Ashton, "That 'Internet of Things' Thing," *RFID Journal*, vol. 22, no. 7, pp. 97–114, 2009.

[5] R. Faludi, *Building Wireless Sensor Networks: with ZigBee, XBee, Arduino, and Processing*, Sebastopol: O'Reilly Media, Inc., 2011.

[6] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, *MQTT Version 5.0 OASIS Standard*, OASIS Open, 2019.

[7] Vercel Inc., *Next.js App Router and React Server Components Documentation*, San Francisco: Vercel, 2024. [Daring]. Tersedia: https://nextjs.org/docs.

[8] R. Cabello (Mr.doob) et al., *Three.js JavaScript 3D Library Documentation*, 2024. [Daring]. Tersedia: https://threejs.org.

[9] B. Benoit, *ArduinoJson: Efficient JSON serialization and parsing for embedded C++*, Version 7, 2024. [Daring]. Tersedia: https://arduinojson.org.

LAMPIRAN

7.3 Lampiran A: Kode Sumber Firmware Lengkap ESP32 (Arduino C++)

Berikut adalah kode sumber program tertanam (*embedded firmware*) yang telah berhasil dikompilasi, diunggah, dan diverifikasi kinerjanya pada perangkat keras fisik ESP32:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// Konfigurasi Pin LCD I2C ESP32
const int LCD_SDA_PIN = 21;
const int LCD_SCL_PIN = 22;

#define LCD_ADDRESS 0x27
LiquidCrystal_I2C lcd(LCD_ADDRESS, 16, 2);

// Konfigurasi Pin Sensor HC-SR04
const int TRIG_PIN = 5;
const int ECHO_PIN = 18;

// Konfigurasi Pin Aktuator LED dan Buzzer
const int LED_HIJAU   = 25;
const int LED_KUNING  = 26;
const int LED_MERAH_1 = 27; // Merah Kiri
const int LED_MERAH_2 = 33; // Merah Tengah
const int LED_MERAH_3 = 32; // Merah Kanan
const int BUZZER_PIN  = 14;

// Parameter Wadah Acuan dan Ambang Batas
const float TINGGI_REFERENSI_CM = 100.0;
const float THRESHOLD_WARNING   = 50.0;
const float THRESHOLD_DANGER    = 80.0;

// Penjadwalan Waktu Non-blocking (millis)
const unsigned long SENSOR_INTERVAL        = 500;  // Baca sensor
(2 Hz)
const unsigned long LCD_ANIMATION_INTERVAL = 150;  // Kedip strobo
LCD
const unsigned long LED_ANIMATION_INTERVAL = 100;  // Kecepatan
lari 3 LED
const unsigned long PUBLISH_INTERVAL       = 2000; // Kirim HTTP
POST (2s)

// Kredensial Jaringan Wi-Fi & Endpoint Backend
```

```
const char* WIFI_SSID     = "JEV AQUATIC_4G";
const char* WIFI_PASSWORD = "jevaquatic12345";
const char* BACKEND_URL   = "http://192.168.1.8:8080/api/
readings";
const char* DEVICE_ID     = "smartflood-esp32-01";

unsigned long lastSensorRead = 0;
unsigned long lastLCDUpdate  = 0;
unsigned long lastLEDUpdate  = 0;
unsigned long lastPublish    = 0;

float distanceCm   = 0;
float waterLevelCm = 0;
bool sensorValid   = false;

enum WaterStatus { SAFE_STATE, WARNING_STATE, DANGER_STATE,
SENSOR_ERROR_STATE };
WaterStatus currentStatus = SENSOR_ERROR_STATE;

const char* statusName(WaterStatus status) {
  switch (status) {
    case SAFE_STATE:    return "AMAN";
    case WARNING_STATE: return "SIAGA";
    case DANGER_STATE:  return "BAHAYA";
    default:            return "ERROR";
  }
}

const char* backendStatus(WaterStatus status) {
  switch (status) {
    case SAFE_STATE:    return "SAFE";
    case WARNING_STATE: return "WARNING";
    case DANGER_STATE:  return "DANGER";
    default:            return "INVALID";
  }
}

void lcdPrintLine(int row, String text) {
  if (text.length() > 16) text = text.substring(0, 16);
  while (text.length() < 16) text += ' ';
  lcd.setCursor(0, row);
  lcd.print(text);
}

bool readWaterLevel() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(3);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
```

```
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL);
  if (duration == 0) return false;

  float measuredDistance = duration / 58.0;
  if (measuredDistance > 400.0) return false;

  distanceCm = measuredDistance;
  if (measuredDistance <= 2.0) {
    waterLevelCm = TINGGI_REFERENSI_CM;
    return true;
  }

  waterLevelCm = TINGGI_REFERENSI_CM - measuredDistance;
  if (waterLevelCm < 0) waterLevelCm = 0;
  if (waterLevelCm > TINGGI_REFERENSI_CM) waterLevelCm =
TINGGI_REFERENSI_CM;
  return true;
}

WaterStatus determineStatus(float level) {
  if (level >= THRESHOLD_DANGER) return DANGER_STATE;
  if (level >= THRESHOLD_WARNING) return WARNING_STATE;
  return SAFE_STATE;
}

void updateActuators() {
  if (currentStatus == SAFE_STATE) {
    digitalWrite(LED_HIJAU, HIGH);
    digitalWrite(LED_KUNING, LOW);
    digitalWrite(LED_MERAH_1, LOW);
    digitalWrite(LED_MERAH_2, LOW);
    digitalWrite(LED_MERAH_3, LOW);
    noTone(BUZZER_PIN);
  } else if (currentStatus == WARNING_STATE) {
    digitalWrite(LED_HIJAU, LOW);
    digitalWrite(LED_KUNING, HIGH);
    digitalWrite(LED_MERAH_1, LOW);
    digitalWrite(LED_MERAH_2, LOW);
    digitalWrite(LED_MERAH_3, LOW);
    noTone(BUZZER_PIN);
  } else if (currentStatus == DANGER_STATE) {
    digitalWrite(LED_HIJAU, LOW);
    digitalWrite(LED_KUNING, LOW);
    tone(BUZZER_PIN, 1000);
    int ledFrame = (millis() / LED_ANIMATION_INTERVAL) % 4;
    if (ledFrame == 0 || ledFrame == 2) {
      digitalWrite(LED_MERAH_1, LOW);
```

```
      digitalWrite(LED_MERAH_2, HIGH);
      digitalWrite(LED_MERAH_3, LOW);
    } else if (ledFrame == 1) {
      digitalWrite(LED_MERAH_1, HIGH);
      digitalWrite(LED_MERAH_2, LOW);
      digitalWrite(LED_MERAH_3, LOW);
    } else if (ledFrame == 3) {
      digitalWrite(LED_MERAH_1, LOW);
      digitalWrite(LED_MERAH_2, LOW);
      digitalWrite(LED_MERAH_3, HIGH);
    }
  } else {
    digitalWrite(LED_HIJAU, LOW);
    digitalWrite(LED_KUNING, LOW);
    digitalWrite(LED_MERAH_1, HIGH);
    digitalWrite(LED_MERAH_2, HIGH);
    digitalWrite(LED_MERAH_3, HIGH);
    noTone(BUZZER_PIN);
  }
}

void updateLCD() {
  if (!sensorValid) {
    lcdPrintLine(0, "SENSOR ERROR!");
    lcdPrintLine(1, "Cek HC-SR04");
    return;
  }
  lcdPrintLine(0, "Jarak: " + String(distanceCm, 1) + " cm");
  if (currentStatus == DANGER_STATE) {
    int blinkFrame = (millis() / LCD_ANIMATION_INTERVAL) % 4;
    char kotak = 255;
    String strobo = String(kotak) + String(kotak) + String(kotak);
    if (blinkFrame == 0)      lcdPrintLine(1, strobo + "  BAHAYA
");
    else if (blinkFrame == 1) lcdPrintLine(1, "     BAHAYA  " +
strobo);
    else if (blinkFrame == 2) lcdPrintLine(1, strobo + "  BAHAYA
" + strobo);
    else                      lcdPrintLine(1, "     BAHAYA     ");
  } else {
    lcdPrintLine(1, "Status: " +
String(statusName(currentStatus)));
  }
}

void setupWiFi() {
  if (WiFi.status() == WL_CONNECTED) return;
  Serial.print("Connecting to WiFi: "); Serial.println(WIFI_SSID);
  WiFi.mode(WIFI_STA);
```

```
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500); Serial.print("."); attempts++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi OK! IP: " + WiFi.localIP().toString());
  } else {
    Serial.println("\nWiFi gagal, retry di loop.");
  }
}

void sendToBackend() {
  if (WiFi.status() != WL_CONNECTED) {
    setupWiFi();
    if (WiFi.status() != WL_CONNECTED) return;
  }
  StaticJsonDocument<256> doc;
  doc["deviceId"] = DEVICE_ID;
  if (sensorValid) doc["distanceCm"] =
serialized(String(distanceCm, 1));
  else             doc["distanceCm"] = -1.0;
  doc["status"]         = backendStatus(currentStatus);
  doc["sensorValid"]    = sensorValid;
  doc["wifiConnected"]  = (WiFi.status() == WL_CONNECTED);
  doc["uptimeSeconds"]  = millis() / 1000;

  char buffer[256];
  serializeJson(doc, buffer);
  Serial.print("POST -> "); Serial.println(buffer);

  HTTPClient http;
  http.begin(BACKEND_URL);
  http.addHeader("Content-Type", "application/json");
  http.setTimeout(3000);
  int code = http.POST(buffer);
  if (code > 0) Serial.println("Backend response: " +
String(code));
  else          Serial.println("HTTP error: " + String(code));
  http.end();
}

void setup() {
  Serial.begin(115200);
  Wire.begin(LCD_SDA_PIN, LCD_SCL_PIN);
  Wire.setClock(100000);
  delay(100);

  lcd.init();
```

```
  lcd.backlight();
  lcd.clear();
  lcdPrintLine(0, "SMART FLOOD K3");
  lcdPrintLine(1, "Sistem Menyala..");
  delay(1500);
  lcd.clear();

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_HIJAU, OUTPUT);
  pinMode(LED_KUNING, OUTPUT);
  pinMode(LED_MERAH_1, OUTPUT);
  pinMode(LED_MERAH_2, OUTPUT);
  pinMode(LED_MERAH_3, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(TRIG_PIN, LOW);
  updateActuators();
  setupWiFi();
}

void loop() {
  unsigned long now = millis();
  if (now - lastSensorRead >= SENSOR_INTERVAL) {
    lastSensorRead = now;
    sensorValid = readWaterLevel();
    currentStatus = sensorValid ? determineStatus(waterLevelCm) :
SENSOR_ERROR_STATE;
  }
  if (now - lastLEDUpdate >= LED_ANIMATION_INTERVAL) {
    lastLEDUpdate = now;
    updateActuators();
  }
  if (now - lastLCDUpdate >= LCD_ANIMATION_INTERVAL) {
    lastLCDUpdate = now;
    updateLCD();
  }
  if (now - lastPublish >= PUBLISH_INTERVAL) {
    lastPublish = now;
    sendToBackend();
  }
}
```

7.4 Lampiran B: Contoh Respons Server Backend dan Payload Telemetri

Data yang dipublikasikan oleh ESP32 setiap 2 detik diterima oleh server Go dan menghasilkan respons status `200 OK` atau `201 Created` dengan contoh log serial:

```
POST -> {"deviceId":"smartflood-
esp32-01","distanceCm":35.2,"status":"SAFE",

"sensorValid":true,"wifiConnected":true,"uptimeSeconds":142}
Backend response: 200
```